Git And Data Safety

. Multi-upstream push is the norm

Every project hosted on multiple Git providers (GitHub primary, GitLab / GitFlic / GitVerse / Bitbucket / Gitea mirrors) MUST push every commit to ALL configured upstream remotes. A commit that lives on only one remote is a future operational risk: when that one provider has an outage, the project is unreachable.

The constitution submodule itself ships an `install_upstreams.sh` helper (and an `Upstreams/` directory with one declaration script per remote). Running it from the submodule root configures all the remotes locally. Consuming projects SHOULD do the same for their own multi- remote topology.

. Submodule changes propagate through submodule commits first

When a change lands in any submodule of the project:

1. **Commit inside the submodule first** (the submodule's own `git add` + `git commit`), since the parent project's commit wrapper does not commit submodule source — it only captures the updated submodule pointer.
2. **Push the submodule commit** to **all** remotes of that submodule.
3. **Then** run the parent project's commit wrapper to capture the updated submodule pointer in main and push main.

Skipping step 1 produces parent commits / tags that point at old submodule HEADs without the actual source.

. Every tag on the main repo MUST be mirrored on every owned submodule

When a version tag is created on the main repo at a specific commit, the same tag MUST be created on **every owned submodule** at that submodule's currently-pointed-to HEAD, and pushed to **every remote** of every owned repo. Third-party submodules (libraries not under the project's control) are excluded. The consuming project's Constitution declares its owned-submodule set explicitly.[5]

. Changelog discipline and multi-format export

Every tagged release MUST ship:

1. A new `docs/changelogs/<tag>.md` entry describing user-visible changes, developer-visible changes, and known caveats.
2. Exports of that entry to `docs/changelogs/<tag>.{html,json,txt}` so external tooling (release portals, ticketing systems, package indices) can consume without needing a Markdown parser.
3. An update to a cumulative `docs/changelogs/CHANGELOG.md` (reverse chronological).

The project SHOULD provide a script (`scripts/testing/export_changelog.sh <tag>` or equivalent) that wraps `pandoc` + `jq` + plain-text Markdown strip.[6]

. Documentation up to the nano-details

Every new feature, fix, or infrastructure change MUST update:

1. The project's CLAUDE.md / AGENTS.md Applied Fixes table (one row, one commit, one release tag).
2. `docs/guides/` for any user-visible or developer-reachable behaviour change.

3. Architecture diagrams, flowcharts, and any reference docs that touch the changed subsystem.
4. The per-version changelog (§5).

Documentation drift after a fix is itself a Constitution violation: undocumented behaviour change is the same defect class as un-tested behaviour change.[7]

. Making false-success results literally impossible

All validation output MUST satisfy the following invariants:

1. Every gate reports concrete PASS / FAIL / SKIP with explicit reason text. A gate that just says "ok" is non-compliant.
2. Every SKIP reason MUST be mechanically distinguishable from a PASS. Summary counters track them separately.
3. FAILs are counted towards exit status. A script that hides FAILs in logs and returns 0 is a bug.
4. Meta-tests (§1.1) catch any gate that always PASSes regardless of condition.
5. Runtime results MUST be cross-referenced against host-side evidence (capture rig[9], screen recording, log analyzer, or equivalent) when the system's own introspection cannot directly verify the behaviour.

§ . Mandatory safety protocol for destructive operations

Every destructive operation MUST execute, in strict order:

1. **Full backup before touching anything.** Hardlinked mirror of `.git` to a sibling backup directory:

   ```
   cp -al .git <backup>/<repo>.git.mirror
   ```

This is near-instant on the same filesystem and uses zero additional disk. If `.git` is small, also create `git bundle create <backup>/main.bundle --all`.

2. **Record critical metadata into the backup dir**:
   - `git show-ref > refs.txt`
   - `git tag > tags.txt`
   - `git submodule status > submodules.txt`
   - `git rev-parse HEAD > head.txt`
   - `git rev-parse HEAD^{tree} > head_tree.txt`
   - `git ls-tree -r HEAD --full-tree | sha256sum > head_tree_content.sha256`
   - `git ls-tree -r HEAD --full-tree > head_tree_listing.txt`
3. **Identify the target state**: the expected HEAD commit, tree hash, and tree content hash after the operation completes. The operation MUST preserve these unless it is explicitly changing HEAD content.
4. **Run the operation** (filter-repo, rebase, etc.). NEVER use `--no-verify` / bypass-hooks flags. NEVER auto-force on failure.
5. **Post-operation verification gate** (all MUST pass):
   - HEAD tree hash matches pre-op target (content identical for non-content-changing rewrites)
   - HEAD tree content sha256 matches pre-op sha256
   - All tags from `tags.txt` are preserved and resolve to valid commits
   - All submodule pointers from `submodules.txt` match current state
   - Project's pre-build / pre-merge gates pass with zero FAIL and zero unclassified WARN
6. **Only if every check in §9.1.5 passes** may the operation be considered successful. Otherwise: restore from backup immediately (`rm -rf .git && mv <backup>/<repo>.git.mirror .git`).
7. **Then and only then** may a force-push proceed — and still not without explicit user authorization per §9.2.

§ . Force-push requires explicit user authorization every time

Force-pushing (including `push --force`, `push --force-with-lease`, or any divergence-recovery code path inside an automated wrapper) overwrites the remote's view of history. On any shared branch this is irreversible.

Rules:

- Automated push wrappers MUST NOT auto-force-push as a fallback to any kind of failure (size rejection, divergence, server error, anything). The historical divergence-recovery code that silently force-resets the remote is forbidden and must be removed or gated behind an explicit flag.
- A human must affirmatively say "force-push" (or equivalent) in the session to authorize each force-push.
- Force-push to `main` (and any equivalent primary branch on owned submodules) is only authorized after the §9.1.5 gate has passed.
- Every force-push event is recorded in `docs/changelogs/<tag>.md` under a "Force-push audit" section (target refs, backup location, validation gate output).

§ . Hardlinked backup is the standard — there is no excuse

Because hardlinked `.git` copies are near-instant and use zero additional disk (same-filesystem inodes), the cost of making a backup is trivial. Any destructive operation without a fresh hardlinked backup is a Constitution violation regardless of how small the repo is or how confident the operator.

§ . Commit-message audit trail for history rewrites

After any `git filter-repo` run (or equivalent), commit a record under `docs/changelogs/` (or `docs/history-rewrites/`) containing:

- What was stripped and why
- Pre-op `.git` size vs post-op `.git` size
- Pre-op HEAD commit vs post-op HEAD commit

- Pre-op HEAD tree hash vs post-op HEAD tree hash (MUST match for non-content-changing rewrites)
- Backup location

10

---

§ . Enforcement

Any commit, tag, or release that violates §1–§9 is non-compliant. The fix is to amend/revert and re-land with compliance. No exceptions for speed pressure. Data-safety violations (§9) and host-session-safety violations (§12) are treated as catastrophic and block the entire release cycle until fully remediated.

11

---

11 4 10

§ . End-user quality covenant

§ . . — Credentials-handling mandate

Forensic anchor — direct user mandate:

> "Credentials or any secret and sensitive data MUST NOT leak! This MUST BE added as the mandatory constraint into all Submodules and main project's Constitution, CLAUDE.MD and AGENTS.MD."

All credentials, secrets, API tokens, passwords, phone numbers, OAuth refresh tokens, signing keys, encryption keys, and any other authentication-or-authorization-bearing data used by test scripts, build automation, or any project tooling MUST be handled per the following rules without exception:

1. **No tracked-file storage.** Credentials NEVER live in any file that git tracks. Placeholder templates with `<replace ...>` or `EXAMPLE_VALUE_DO_NOT_USE` placeholders ARE allowed in `.example` files committed to show operators what keys to populate.

2. **Repository-wide ignore.** `.gitignore` MUST exclude all credential-bearing paths:
   - `.env`, `.env.*`, `*.env`, `.<service>.env`, `*.<service>.env`
   - `scripts/testing/secrets/*` with `.example` + `README.md` exception
3. **Runtime-load-only.** Tests load credentials AT EXECUTION TIME from operator-populated files under `scripts/testing/secrets/` (or per-project equivalent). If the file is missing, the test SKIPs with "credentials not configured for " — it does NOT proceed without credentials and does NOT use defaults.
4. **No echo / no log.** Test scripts MUST NEVER print, log, or include credentials in error paths, stack traces, screen recordings, or output artifacts.
5. **Per-service file separation.** One file per service keeps blast radius small.
6. **Filesystem permissions.** `.env` files are `chmod 600`, parent directory is `chmod 700`.
7. **Rotation on suspected leak.** Rotate at provider, update local files, audit captured artifacts.

The consuming project's `.gitignore` MUST be verified before every commit: `git ls-files --cached | grep -E "\\.env$"` MUST show no real `.env` files (only `.env.example`).

§ . . .A — Pre-store credential leak audit (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-05-17):

> "Us these for all future testing (full automation testing) and make sure they are not leaking anywhere or get git versioned!"
>
> [Discovered during execution: the operator-provided test credentials ALREADY existed in 5 tracked files dating to prior project cycles. §11.4.10 + §11.4.30 catch new commits but did not detect the pre-existing leak when the operator re-provided the same values for a new use.]

When an operator provides credentials, API tokens, signing keys, or any other secret material to be stored in the project's gitignored configuration (.env, scripts/testing/secrets/, *secrets/*, etc.), the agent or human storing them MUST FIRST execute a repo-wide audit for prior leaks of THOSE specific values BEFORE storing.

The audit MUST:

1. **Grep every tracked file** for the literal credential value(s). `git ls-files | xargs grep -l <value>` is the canonical form. Search MUST cover all file types — `.md`, `.json`, `.yaml`, `.kt`, `.go`, `.py`, `.sh`, `.sql`, build configs, docs.
2. **Grep the entire git history** for the literal credential value(s). `git log -S<value> --all --source --remotes` reveals historical commits even if the current tree is clean. A history-only leak is a §11.4.10 violation of equal severity to a tree-leak — anyone who pulled an older commit still has the value in their working copy of any branch they cloned.
3. **Report findings to the operator BEFORE storing the new value in any operator-controlled location.** The operator may then decide whether to (a) rotate the credential at the upstream provider before reusing, (b) accept the historical leak as a known-compromise and proceed, or (c) abort the storage. Storing the value without surfacing the audit is itself a §11.4 PASS- bluff at the security layer.
4. **If a leak is found:** open a forensic incident record per the project's §6/§7 sixth-law-incidents discipline (or equivalent), redact the literal values from all tracked files in-place (replace with `<redacted-per-§11.4.10>` placeholders), and record an OPERATOR ACTION REQUIRED for rotation per §11.4.10 sub-clause 7. The redaction commit lives on master IMMEDIATELY; the history-purge follow-up (git-filter-repo / BFG / equivalent + force-push to every upstream) requires explicit per-operation operator authorization per §9.2.
5. **Strengthen pre-push hook detection** when a previously-undetected class of literal credential is discovered. The pre-push hook's credential-pattern grep MUST be extended to catch the specific pattern class that escaped (literal usernames matched with literal passwords in the same file, well-known credential SHA-256 hashes, org-specific naming conventions for service accounts, etc.). The extension MUST land in the same commit as the redaction.

Pre-build gate (recommended) `CM-PRE-STORE-CREDENTIAL-AUDIT` (when implemented in consuming project) checks the commit message body of any commit that touches `.env`, `.env.example`, `scripts/testing/ secrets/`, or equivalent for an `Audit-Pre-Stored:` stamp citing the audit's outcome. Paired mutation (§1.1) strips the stamp and asserts gate FAILs.

Composes with §11.4.10 (the core credentials mandate this extends), §11.4.30 (.gitignore + no-versioned-artifacts — overlapping enforce- ment surface), §11.4.6 (no-guessing — audit findings stated as fact or as PENDING_FORENSICS), §11.4.7 (demotion-evidence — "I think it's not leaked" without grep evidence is a guess). Classification: universal (per §11.4.17). No escape hatch.

2026 05 15

§ . . — .gitignore + No-Versioned-Build-Artifacts Mandate (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-05-15):

> "every project module, every Submodule, every servcie and apolication MUST HAVE proper .gitignore file! We MUST NOT git version build artifacts, cache files, tmp files, main .env file(s) or any files containing sensitive data, API keys or token! Any build derivate which we can recreate by executing proper mechanism for generating MUST NOT be versioned! We MUST pay attention what is going to be commited every time we are preparing to execute commit! If any violetion is detected it MUST be fixed before commit is executed!"

Operative rule. Every project module, owned-by-us submodule, service, and application under this Constitution MUST ship a proper `.gitignore` covering its full set of forbidden-from-version- control patterns. The following file/directory classes MUST NEVER appear in version control:

1. **Build artefacts** — anything produced by a build / compile / package step that can be regenerated from sources:
   - Binaries (`/bin/`, `/build/`, `/dist/`, `/out/`, `target/`, `*.exe`, `*.dll`, `*.so`, `*.dylib`, `*.a`, `*.o`, `*.class`, `*.pyc`).

    - Generated source files where the generator is committed (e.g. protobuf `.pb.go` may be checked in OR ignored — project decides — but never both).
    - Bundled assets where the bundler is committed.
2. **Cache files** — anything a tool regenerates on demand:
    - `__pycache__/`, `.pytest_cache/`, `.mypy_cache/`, `.ruff_cache/`, `node_modules/`, `.next/`, `.nuxt/`, `.cache/`, `.gradle/`, `.idea/cache/`, `.vscode-test/`, `target/`, `.terraform/`, language-server caches.
3. **Temp files** — `*.tmp`, `*.swp`, `*.swo`, `*~`, `.DS_Store`, `Thumbs.db`, IDE backup files, `*.orig`, `*.rej`.
4. **Sensitive-data files** — anything containing credentials, tokens, keys, or personal data:
    - `.env`, `.env.*` (allow `.env.example` / `.env.sample` / `.env.template` with placeholder values only — never real secrets even as examples).
    - `*.pem`, `*.key`, `*.crt`, `*.p12`, `*.pfx`, `id_rsa*`, `id_ed25519*`, `*.kdbx`, `.netrc`, `secrets/` directory trees.
    - `api_keys.sh`, any file containing `BEGIN PRIVATE KEY`, credential tokens, or session cookies.
5. **Generated reports / logs** — `*.log`, `coverage.out`, `*.coverage`, `htmlcov/`, `*.gcda`, `*.gcno`, screenshots/ recordings that aren't reference assets.
6. **OS / IDE / personal-state files** — `.idea/`, `.vscode/` (except shareable `.vscode/settings.json` etc. if explicitly project-shared), `.history/`, `.svn/`, editor-state files.
7. **Test playback artefacts** (§11.4.14) — every test's runtime capture goes under an ignored evidence directory unless the capture IS the reference asset.

Anti-bluff invariant. Per CONST-035 / §11.4 the presence of a `.gitignore` line alone is not sufficient — the project MUST also verify that no file matching the forbidden patterns is currently tracked. A `.gitignore` that lists `*.log` while `app.log` sits tracked in the repo is a §11.4.30 violation of equal severity to no `.gitignore` at all.

Pre-commit attention. Every commit author (human OR agent) MUST inspect `git diff --staged` and `git status` BEFORE executing the commit. If a staged path matches any forbidden class, the commit MUST be aborted and the issue fixed (un-stage the path, add to `.gitignore`, scrub if already-tracked). Gate `CM-GITIGNORE-PRECOMMIT-AUDIT`: pre-commit hook (or commit wrapper in §2) inspects every staged path against the forbidden-pattern matrix; hits abort the commit with a directed error pointing the operator at the specific offending path and the canonical fix. Paired mutation (§1.1): stage a `*.log` → gate FAILs.

Cascade reach. This rule applies recursively to every owned-by- us submodule per CONST-047 and to every owned-submodule's nested non-owned trees within their working copy. The `.gitignore` files themselves are project-specific in their concrete patterns but universally bound to the rule's normative force.

Secret-leak intersection. §11.4.30 composes tightly with §11.4.10 (Credentials-handling mandate / CONST-042) and §12.1. A `.env` leak is BOTH a §11.4.30 violation (build/sensitive file versioned) AND a §11.4.10 violation (credential leak). The combined severity is **release-blocker requiring rotation + post-mortem** per §11.4.10.

Coverage of "recreatable" content. When in doubt about whether something is a build derivative: if there exists a documented, scripted, or automated mechanism that recreates the file from sources, the file is a build derivative and MUST be ignored. The committed sources MUST include the generator (Makefile target, npm script, codegen invocation), so any downstream consumer regenerates the artefact on demand.

Classification: universal (per §11.4.17). No escape hatch beyond the explicit exceptions enumerated above (e.g., `.env.example` placeholder files). Severity-equivalent to a §11.4 PASS-bluff at the repository-hygiene layer — a tracked build artefact silently drifts vs. its source over time, producing the same class of "works-on-my-machine" surprise that the §11.4 anti-bluff covenant forbids at the test layer.

Composition. §11.4.30 composes with §1 (test coverage — ignored files don't get spuriously included in coverage stats), §2 (commit-wrapper enforces the pre-commit gate), §9.1 (destructive-operation safeguards — never `git clean` an operator's working tree to "fix" a violation), §11.4.10 (credentials-handling), §11.4.12 (auto-generated docs sync — generators committed, outputs ignored only if recreatable on demand from the same generator), §11.4.17 (universal — applies to every consuming project), §11.4.18 (script docs — generator scripts documented), §11.4.20 (subagent delegation for cross-submodule .gitignore audit sweeps), §11.4.25 (coverage ledger lists `.gitignore` presence per submodule), §11.4.26 (constitution-update workflow — the constitution submodule's own `.gitignore` complies), §11.4.27 (no-fakes: test fixtures that ARE the reference asset are tracked; runtime captures are ignored), §11.4.28 (submodules-as-equal-codebase: each submodule's `.gitignore` audited on equal basis), §11.4.29 (the renamed paths in the lowercase migration MUST also be properly ignored where applicable), CONST-047 (recursive governance reach).

2026-05-15

§11.4.31 — Mandatory install_upstreams on clone/add Mandate (User mandate, 2026-05-15)

Forensic anchor — verbatim user mandate (2026-05-15):

> "Every Submodule or Git repository we add or clone MUST BE upstreams installed using Upstreamable utility which MUST BE available through exported paths of the host system (in .bashrc or .zhrc) using install_upstreams command executed from the root of the cloned (added) repository - only if in it is Upstreams or upstreams directory present with bash script files (recipes) for all repository's upstreams!"

Operative rule. Every time an operator or agent adds or clones a Git repository (`git clone`, `git submodule add`, `incorporate-submodule` per §11.4.31, etc.) under any consuming project of this Constitution, the post-clone procedure MUST be:

1. `cd` into the newly-cloned (or newly-added submodule's) working tree.

2. If the working tree contains an `upstreams/` directory (or legacy `Upstreams/` per the §11.4.29 transition) populated with one or more `*.sh` recipe files declaring upstream Git SSH URLs (the canonical declaration format defined by this constitution submodule's `Upstreams/*.sh` shape), the operator or agent MUST invoke `install_upstreams` from that directory.
3. `install_upstreams` is a host-system utility installed on the operator's `PATH` via `.bashrc` or `.zshrc` export. Implementation lives in the constitution submodule (`install_upstreams.sh`); operators alias / symlink it onto `PATH` once per host setup. The utility reads the recipe files, configures every declared upstream as a named git remote, and fans out `origin` push URLs across all declared upstreams.
4. If no `upstreams/` directory is present, no `install_upstreams` invocation is required.
5. If `upstreams/` is present but `install_upstreams` is not on `PATH` , the operator MUST install it (per the constitution submodule's setup docs) BEFORE making the clone usable.
6. Skipping step 2 when an `upstreams/` directory IS present is a §11.4.36 violation — the next push from that working tree will land on only one upstream, breaking §2.1 (Multi-upstream push is the norm).

Pre-commit attention. Before the first commit lands inside the newly-cloned working tree, the operator MUST verify that `install_upstreams` has been executed (when applicable). The quickest check:
`git remote -v | grep -c push` reports the expected upstream count. If it reports `1` while `upstreams/*.sh` declares more, abort the commit and run `install_upstreams` first.

Automation. The constitution submodule's tooling (the future `incorporate-submodule` per §11.4.31, the existing `scripts/init-submodules.sh` patterns in consuming projects) MUST auto-invoke `install_upstreams` as part of any clone / add operation when the target tree has an `upstreams/` directory. Operator-explicit manual invocation remains supported.

Gate `CM-INSTALL-UPSTREAMS-ON-CLONE` . Pre-merge gate inspects every owned-by-us submodule pointer commit and verifies that: (a) if `upstreams/` (or legacy `Upstreams/`) is present in the submodule's tree, (b) the submodule's local

checkout has at least as many configured push URLs as the declared recipe count. Paired mutation (§1.1): remove one upstream from local `origin --push` config → gate FAILs.

Composition. §11.4.36 composes with §2 (single-entrypoint commit/push wrapper), §2.1 (multi-upstream push is the norm — this rule is its setup-time complement), §3 (submodule changes propagate through submodule commits first — requires multi-upstream parity), §9.2 / CONST-043 (no force-push — the multi-upstream wrap-up makes unforced parity easy), §11.4.17 (universal — every consuming project's clone procedure), §11.4.20 (subagent delegation when batch-cloning), §11.4.28 / CONST-051 (owned-submodule discipline), §11.4.29 / CONST-052 (`Upstreams/` → `upstreams/` transition is exactly this rule's scope), §11.4.30 / CONST-053 (`.gitignore` ignores `upstreams/` only if the project explicitly decides not to track recipes, which is unusual — recipes ARE source of truth and SHOULD be tracked), §11.4.31 / CONST-054 (submodule-dependency-manifest works alongside upstreams recipes: helix-deps.yaml lists WHAT to add at parent root, upstreams/ recipes list WHERE to push the resulting clones).

Classification: universal (per §11.4.17). No escape hatch. Severity-equivalent to §2.1 multi-upstream-push-violation at the clone-time setup layer — without the post-clone install, the working tree silently has a single-upstream blind spot.

2026 05 15

---

§ . . — Fetch-before-edit mandate (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-05-15):

> "Make sure that feedback_fetch_before_edit memory rule is part of our constitution Submodule - the root Consitution, AGENTS.MD and CLAUDE.MD. Validate and verify that Proejct-Toolkit and all Submodules do inherit all of them! Follow the constitution Submodule documentation for details."

Background. In multi-agent / multi-upstream codebases — where parallel Claude Code, Cursor, Aider, Codex, Gemini CLI, or human operator sessions may operate on overlapping scope — the local working tree's state can lag behind the canonical upstream state by the time any given agent receives a task. Acting on stale local state produces three failure modes documented in the originating session (2026-05-15):

1. **Redundant work** — the agent re-does what a parallel session already finished, wasting tokens and operator review time. (In the originating incident, "Gitee removal" had already been committed upstream by a parallel agent 25 minutes earlier; the second agent's `git remote remove gitee` was a no-op echo of work already done.)
2. **False confidence** — the agent reports completion of work that was already done by someone else, with no mechanical signal that their commit duplicates upstream history.
3. **Divergent history** — if the agent commits a parallel "completion" of already-done work, the result is two siblings of the same change, doubling the conflict surface for the next push attempt to multi-upstream remotes (§2.1).

The mandate. The FIRST git-touching action of any session, on any consuming project that participates in this constitution, MUST be:

```
git fetch --all --prune
git log --oneline HEAD..@{u}            # parent
git submodule foreach --recursive 'git fetch --all --prune --quiet'
```

If `HEAD..@{u}` is non-empty, the agent MUST integrate (ff-merge, rebase, or — if non-fast-forward — surface to operator per §11.4.4) BEFORE issuing any local edit, scanner run, or test cycle. The fetch step is non-negotiable even when the operator's directive is phrased as "do X immediately" — the 30-second check prevents hour-long conflict reconciliation later.

Scope. Applies to:

1. The consuming project root.
2. Every owned submodule (per §11.4.28) — recursive.

3. The constitution submodule itself (§11.4.26 step 1 makes this explicit for constitution-side edits; §11.4.37 generalises it to ANY edit on the consuming project, not only constitution edits).
4. Any dependency cloned via `incorporate-submodule` (§11.4.31) or `git submodule add` (§11.4.36).

Anti-bluff invariant. The fetch+log check MUST produce captured evidence — the actual `HEAD..@{u}` output, even if empty. Skipping the check on the basis of "I just fetched" or "nothing could have changed in the last N minutes" is a §11.4.6 (no-guessing) violation: the remote state is not knowable without a fetch.

Composition. Composes with §2.1 (multi-upstream push — without the fetch, the agent can't know which upstream has the canonical ref), §11.4.4 (test-interrupt-on-discovery — newly-fetched commits that contradict the planned work are exactly the "freshly discovered defect" that triggers cycle interruption), §11.4.6 (no-guessing — remote state requires fetch, not assumption), §11.4.20 (subagent delegation — parallel subagents MUST coordinate through fetched state, never assumed-local state), §11.4.26 (constitution update workflow — this rule generalises §11.4.26 step 1 to ALL edits, not only constitution-file edits), §11.4.32 (post-constitution-pull validation — fetch-before-edit happens BEFORE the constitution-pull sweep §11.4.32 mandates).

Gates. Pre-build gate `CM-FETCH-BEFORE-EDIT-AUDIT` (when implemented in the consuming project) audits the most-recent commit range against the upstream HEAD at the commit's parent — if the parent ref was not the upstream HEAD at the time the commit was authored, FAIL. Paired mutation (§1.1): synthetic commit whose parent is N commits behind the then-current upstream HEAD — gate must FAIL.

Classification: universal (per §11.4.17). The rule has no project-specific assumptions — it applies to any multi-upstream / multi-agent codebase. No escape hatch. Severity-equivalent to a §11.4 PASS-bluff at the planning layer — acting on stale local state is the operational analog of asserting truth from unverified premises.

§ . . — Pre-Force-Push Merge-First Mandate (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-05-17):

> "make sure we bring everything from branches to our side before forc push is done! Afer everything is safely and fully merged and all potential conflicts (if any) resolved, then do force push! make sure nothing isnlost, broken or corrupted on bith sides! add these rules in our root Constitution, CLAUDE.MD, AGENTS.MD (constitution Submodule) if itnis not added already! Extremely important rules and mandatory constraints we MUST HAVE and fully respect!"

Operative rule. Any force-push (`git push --force` , `git push --force-with-lease` , `git push +<ref>` , equivalent history-rewriting operation on any remote) authorised under §9.2 / CONST-043 MUST be preceded by a mechanical 4-step merge- first pipeline that brings every remote-side commit into the local tree, resolves every conflict carefully, and verifies nothing is lost or corrupted on EITHER side BEFORE the overwriting push is executed.

The 4-step pipeline (mandatory, in order):

1. **Fetch every remote-side reference.** Run `git fetch --all --prune --tags` against every configured remote (origin + every upstream). Capture the output.
2. **Integrate every divergent commit locally.** Compute `git log --oneline HEAD..<remote>/<branch>` for every remote. For every non-empty range, integrate the remote commits into the local tree via the appropriate strategy:
   - **`git rebase <remote>/<branch>`** when the local divergent work is a strict superset that should land on top, OR
   - **`git merge <remote>/<branch>`** when the local + remote histories are independent additions that both deserve preservation, OR
   - **operator-confirmed cherry-pick** when remote contains a subset of commits already present locally in different form.

3. **Audit the integrated tree.** Verify (a) no conflict markers anywhere in the working tree (`grep -rn '^<<<<<<< \|^=======$\|^>>>>>>> ' .` returns empty across all governance + source + test files — third-party docs containing the markers as illustrative content excepted), (b) no file silently dropped (`git diff --stat HEAD@{1} HEAD` shows only expected additions), (c) every previously-passing test still passes on the integrated tree (per §11.4.4 + §11.4.40 baseline), (d) every captured- evidence artifact from the pre-merge state still validates.
4. **Execute force-push.** Only after steps 1-3 produce captured evidence of clean integration: `git push --force-with-lease <remote> <ref>` (NEVER `--force` without `--with-lease` unless explicitly authorised per §9.2 sub-clause 6 for a specific remote where lease semantics are unavailable). One force-push event per CONST-043 authorisation — no batch authorisation across multiple force-pushes.

Three failure modes prevented:

(a) **Remote-side content loss.** Force-push without prior fetch silently overwrites commits a parallel session, sibling agent, or co-developer landed on the remote. The lost work is recoverable from the remote's reflog only within its TTL window — typically 30-90 days — but the audit trail (PR comments, CI evidence, governance signatures) is lost permanently.

(b) **Stale-state act on remote.** Force-push from a divergent local state that was branched from a remote tip N hours ago treats the remote-side N hours of work as nonexistent. The operator's `--force-with-lease` safety check catches the literal SHA mismatch but ONLY when invoked AFTER a fetch that updated the lease reference; without step 1, the lease check is reading stale local refs.

(c) **Conflict-driven corruption.** A merge that completes with unresolved markers in the tree (one half kept, other dropped silently because the marker was inside a comment block, OR markers committed verbatim as in CONST-049-prerequisite incidents observed 2026-05-17 on helix_qa + containers) lands broken governance / source on the force-push target. Step 3's explicit conflict-marker grep is the mechanical guard.

Two-gate composition with CONST-043. §11.4.41 does NOT relax CONST-043's operator-approval requirement — it adds a SECOND gate on top:

- **Gate A (CONST-043).** Operator types explicit per-operation force-push authorisation in the conversation.
- **Gate B (§11.4.41).** Agent executes the 4-step merge-first pipeline, captures evidence of clean integration, presents evidence to operator BEFORE the force-push.

Both gates required. CONST-043 alone authorises a force-push that loses remote work; §11.4.41 alone risks force-pushing without operator awareness. The two together produce a force-push that is BOTH operator-authorised AND remote-safe.

Verification artefact. Every §11.4.41-governed force-push emits a `docs/changelogs/<tag>.md` "Force-push merge-first audit" section containing: (i) `git fetch` output, (ii) per- remote `HEAD..<remote>/<branch>` log before integration, (iii) integration strategy chosen per remote with rationale, (iv) post-integration conflict-marker scan output (must be empty), (v) post-integration test suite delta (must show only expected changes), (vi) the `--force-with-lease` push output with lease SHA evidence, (vii) CONST-043 authorisation quote from the conversation.

Cascade requirement (per CONST-047). This anchor (verbatim or by `§11.4.41` / `CONST-061` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to a §11.4 PASS-bluff at the remote-data-integrity layer. Gate `CM-FORCE-PUSH-MERGE-FIRST` walks `docs/changelogs/<tag>.md` "Force-push" entries for the 7 audit elements; paired mutation strips any element and asserts gate FAILs.

Classification: universal (per §11.4.17) — every owned project executes force-pushes against its remotes; the merge- first discipline is reusable verbatim. No escape hatch — the operator-pressure escape ("just force-push, we'll fix it later") is the exact failure mode this anchor closes.

Composes with: §9.2 (data-safety hardlinked backup), §11.4.4 (test-interrupt-on-discovery — broken integration triggers rollback), §11.4.6 (no-guessing — every step's outcome captured, not assumed), §11.4.26 (constitution-submodule update pipeline — that mandate is the per-submodule specialisation of §11.4.41 for governance-files-only commits), §11.4.32 (post-pull validation — the audit step's mechanical companion), §11.4.37 (fetch-before- edit — step 1 enforces it for the

force-push specifically), §11.4.40 (full-suite retest — step 3's test-evidence prerequisite), CONST-043 (per-operation operator approval), CONST-047 (recursive cascade).

§ . . — Pre-Push Fetch + Investigate + Integrate Mandate (User mandate, - -)

Forensic anchor — direct user mandate (verbatim, 2026-05-20):

> "before pushing changes to any upstream for any repository - main repo or Submodule, we MUST fetch and pull all changes. Once these are obtained WE MUST investigate what is different compared to head position we were on last time before fetching and pulling new changes! We MUST understand what is done and for what purpose, easpecially how that does affect our project and our System in general! Any mandatory changes or improvements required by fresh changes we just have brough in MUST BE incorporated, covered with all supported types of the tests which will produce as a result of its success execution REAL PROOFS of working for all componetns and functionalities covered and work fully in anti-bluff manner!"

This anchor is the **everyday-push variant** of §11.4.41 (Pre-Force-Push Merge-First). §11.4.41 governs the destructive force-push case; this anchor governs EVERY push including ordinary fast-forwards. The complementary discipline closes the regression-via-stale-baseline gap: a project committing against a HEAD that's behind any upstream's main risks pushing changes that conflict with, regress, or fail to incorporate accepted work landed in parallel by other consumers (Catalogizer and other consuming projects on the constitution submodule; sibling projects on other shared submodules).

The mandatory 5-step pre-push cycle (every push, every repository — main + every submodule):

1. **Fetch all remotes** — `git fetch --all --prune --tags` (or per-remote loop if `--all` is unavailable). Capture stdout for the audit trail.

2. **Pull all upstream branches** — `git pull --no-rebase <remote> <branch>` for each configured remote whose tip differs from the local branch. Resolve any merge conflict per §11.4.41 step 2 (rebase / merge / cherry-pick per consumer judgment, NOT auto-`--ours` or `--theirs`).
3. **Investigate the diff vs OUR previous HEAD** — `git log --oneline <previous-head>..HEAD` + `git diff --stat <previous-head>..HEAD` + read EVERY commit's body. For each foreign commit understand: (a) what changed, (b) why (forensic anchor + cited user mandate or §-anchor), (c) how it affects OUR project + OUR System (does it touch surfaces we ship? does it introduce constraints we now MUST honor? does it deprecate or supersede patterns we use?).
4. **Integrate mandatory changes + cover with full anti-bluff test coverage** — if the pulled-in work mandates anything (new §-anchor, new gate, new propagation requirement), implement it per §11.4.4(b) four-layer coverage (pre-build gate + post-build inspection + on-device test + HelixQA Challenge) + §11.4.43 TDD-fix RED-test-first discipline. Every PASS MUST carry §11.4.5 captured-evidence (REAL PROOFS — not metadata-only).
5. **Then push** — only after Steps 1–4 land. Push to every configured remote in the per-repo cascade order (canonical first, then mirrors). Verify the push landed with `git ls-remote <remote> <branch>` post-push.

Composition with §11.4.26 (constitution-submodule update pipeline — per-submodule specialisation) + §11.4.32 (post-pull validation) + §11.4.37 (fetch-before-edit) + §11.4.40 (full-suite retest before tag) + §11.4.41 (pre-force-push merge-first — the force-push case of this anchor) + §11.4.42 (iteration discipline) + §11.4.43 (TDD-fix-discipline) + §11.4.4(b) (four-layer coverage) + §11.4.5 (audio + video quality analysis comprehensiveness) + §11.4.6 (no-guessing — every integration decision cites the foreign commit by SHA, not "we think it does X").

No escape hatch — there is no `--skip-fetch`, `--no-investigate`, `--fast-push`, `--trust-upstream` flag. The discipline exists because:

- Pushing-without-fetching produces silent stale-baseline regressions (consumer A's PR diverges from consumer B's PR; neither saw the other; both merge; resulting tree corrupts).

- Pulling-without-investigating destroys the foreign work's traceability (we now carry someone else's code without understanding it; future debugging crosses the §11.4.6 no-guessing line).
- Integrating-without-testing reproduces the §11.4 PASS-bluff pattern (we adopt a change but don't prove it works in OUR system).
- Pushing-after-integration-without-evidence reproduces §11.4 itself (untested integration = release-blocker by definition).

Per-repository scope: this anchor applies to (a) the parent consuming project's repo, (b) the constitution submodule (across all 6 remotes), (c) every owned submodule (presenter, vlc-player, nova-player, mpv-player, gramophone-player, rhythm-player, strep-player, smarttube-player, torrserve, lampa), (d) every nested submodule (smarttube-player/SharedModules / smarttube-player/MediaServiceCore / smarttube-player/MediaServiceCore/SharedModules), (e) every HelixQA dependency (Challenges, Containers, DocProcessor, LLMOrchestrator, LLMProvider, VisionEngine, HelixQA).

Audit-trail requirement: every push event MUST be reconstructable post-hoc from `docs/changelogs/<tag>.md` + the per-repo `git log` evidence. Operators auditing a release MUST be able to: (1) see what each repo's `HEAD..@{u}` was BEFORE this push, (2) see what foreign work was pulled in, (3) see what new tests + captured-evidence were generated by the integration. Anchor enforcement may be partially automated by extending `scripts/commit_all.sh` to refuse pushes without an audit-trail log entry.

Pre-build gate `CM-COVENANT-114-71-PROPAGATION` enforces this anchor literal across the consumer fleet (parent + 10 owned submodules + nested + HelixQA). Paired mutation strips the anchor literal → gate FAILs. Paired meta-test also asserts the audit-trail-log file produced by a recent push exists + lists the foreign-commit SHAs that were pulled in.

Canonical authority: constitution submodule `Constitution.md` §11.4.71.

Non-compliance is a release blocker regardless of context.

§ . . — Working-tree quiescence rule for subagent commits (User mandate, - -)

Short tag: `working-tree quiescence`.

The mandate. No subagent commit may proceed while any concurrent mutation gate, paired-mutation experiment, or other in-flight mutation is live in the same checkout. Before `git add`, the committing agent MUST grep its own working tree for mutation markers (`MUTATED for paired`, `// always pass`, `return json.Marshal` shortcut paths, `// MUTATION` / `# MUTATION` annotations, `_mutated_*` filename suffixes, etc.) and explicitly account for every modified file in the staging area. Any unexplained file in the staging area triggers ABORT.

Forensic anchor (verbatim user mandate, 2026-05-22):

> "no subagent commit may proceed while any concurrent mutation gate is in flight in the same checkout. Before `git add`, the committing agent MUST `grep` its own working tree for mutation markers (`MUTATED for paired`, `// always pass`, `return json.Marshal` shortcut paths, etc.). Any unexplained file in the staging area triggers ABORT."

Lesson (forensic case study). A consuming project's logo-fix subagent (Herald commit `72e81ab`, 2026-05-21) ran in a checkout where a paired §1.1 mutation gate had temporarily introduced an `// always pass` shortcut into `commons_auth/middleware.go` (JWT-bypass mutation, intended to be reverted by the same gate). The subagent's `git add` + `git commit` swept the mutation residue into the same commit as the unrelated logo fix, and the resulting commit was pushed to all four mirrors before any other agent caught it. The fix (Herald `d5bd360`, "SECURITY FIX: restore commons_auth/middleware.go JWT verify") landed within the hour, but the window during which production-equivalent binaries shipped with a bypassed JWT verify is a real security-defect window — small but non-zero — and is the canonical example of why this rule exists.

Operative rule.

1. Subagent (and main-thread) commit flows MUST run a pre-`git add` quiescence check that:
   - Greps the working tree for mutation markers (canonical list above; the consuming project MAY extend with project-specific markers).
   - Lists every modified / untracked / staged file and matches each one against the subagent's declared scope. Any file outside the declared scope → ABORT.
   - Cross-checks `git status --porcelain` against the subagent's task description; unaccounted entries → ABORT.
2. Any active mutation gate (paired-mutation experiment in progress, AVR/AVI mutation cycle, manual `// always pass` patch, etc.) MUST be serialised — the gate runs to completion (mutate → assert FAIL → restore → assert PASS) and the working tree is verifiably clean BEFORE any unrelated commit may proceed in the same checkout.
3. Concurrent subagents working in the SAME checkout (single working tree, no `git worktree add`) MUST coordinate through a lockfile or marker file (e.g. `.git/MUTATION_IN_PROGRESS`) so a logo-fix subagent cannot race a JWT-mutation subagent. The constitution submodule's `scripts/quiescence_check.sh` (when implemented) is the universal lock helper.
4. When parallel work is necessary, the §11.4.20 / §11.4.70 subagent-driven mandate SHOULD be combined with `git worktree add` to give each subagent an isolated working tree — eliminates the cross-mutation race by construction.
5. Post-commit, an automated `mutation-residue-scanner` (consuming project's `scripts/mutation_residue_audit.sh`) MUST run before push. Any commit containing a mutation marker → push BLOCKED, commit MUST be reverted or amended before mirrors are updated.

Composes with §1.1 (mutation-paired gates — quiescence rule protects the mutation cycle from concurrent contamination), §11.4.20 / §11.4.70 (subagent-driven-by-default — quiescence rule is what makes parallel subagent dispatch safe), §11.4.27 (no-fakes-beyond-unit — a mutation residue swept into a commit IS a fake-pass surface in production), §11.4.10 (credentials handling — same class of "do not let unrelated content leak into a commit"), §107 (a security-bypass mutation

that ships to production is the gravest §107 PASS-bluff: tests pass against the mutated codebase, but the end user loses an entire security property), §11.4.71 (pre-push fetch + investigate — quiescence check is the pre-push companion).

Classification: universal (per §11.4.17). Every project under this Constitution carries this rule.

Canonical authority: constitution submodule `Constitution.md` §11.4.84.

Non-compliance is a release blocker. No `--allow-residue`, `--skip-quiescence`, `--mutation-cleanup-later` flag exists. A mutation marker that lands in a tagged commit is a critical defect regardless of how briefly it persisted.

2026 05 26

§ . . — Background-push mandate: commit-lock release immediately after commit, push runs detached (User mandate, - -)

Forensic anchor — verbatim User mandate (2026-05-26):

> "Make sure all these pushes are being done ALWAYS in backgroubd in parallel with main work stream so we do not loose time waiting. Everything is commited anyway? If we can do this add this as mandatory rule / constraint that we must respect and follow ALWAYS! ... We MUST ensure that main work stream has always something to do, or wait for the results only when that is absolutely required! We have enormous amount of work in front of us, short deadlines and we MUST DELIVER!"

Forensic incident. 2026-05-26 between 14:35Z and ≥19:30Z (~5 hours observed), a single `commit_all.sh` invocation held its `.git/.commit_all.lock` flock the entire time — the COMMIT had succeeded within seconds at `63d307234ce`, but the synchronous `do_push` call kept the flock while `git push vasicdigital_gitlab` did a large initial-mirror upload. **Every subsequent commit_all.sh invocation in that window was BLOCKED** — 21 submodule

pointer bumps + parent §11.4.87 propagation work + §KA Critical audio quality matrix artifacts all sat staged in the working tree waiting for the push to finish. This is the exact anti-pattern §11.4.87 (zero-idle) prohibits at the project-execution layer.

The mandate.

(A) Flock release IMMEDIATELY after commit lands. Once `git commit` returns 0 (commit object recorded in the local repo), the `.git/.commit_all.lock` MUST be released BEFORE `do_push` is invoked. The commit is durable on local disk regardless of push outcome — there is no correctness reason to gate further local work on a remote-push round-trip.

(B) Push runs detached. After flock release, the push step is spawned via `nohup ./push_all.sh ... > <log> 2>&1 &` then `disown` so the orchestrator's exit does not propagate SIGTERM. The orchestrator's exit code reports COMMIT success, NOT push success (push success is logged separately and reconciled by §11.4.71 fetch-before-edit on the next interactive turn).

(C) Push concurrency control: per-remote serialization, multi-remote parallelism. `push_all.sh` MUST acquire a per-remote flock (`.git/.push.<remote>.lock`) so two concurrent invocations targeting the same remote serialize (avoid push-race / non-fast-forward), but invocations targeting DIFFERENT remotes run in parallel. This converts the prior 5-hour serial-stall pattern into a multi-stream pipeline where the operator's main work stream never blocks on the slowest mirror.

(D) Failure surface. Backgrounded push failures land in `qa-results/push_failures/<timestamp>_<remote>.log` . The next interactive autonomous-loop tick MUST check that directory (per §11.4.87(A) "no external dependency is in-flight" check) and surface failures — re-push attempted automatically, or operator notified if auth/quota issue. Silent push-failure is a §11.4 PASS-bluff at the distribution layer.

(E) Synchronous-push escape. An explicit `--sync-push` CLI flag on `commit_all.sh` preserves the legacy synchronous behaviour for `--force-with-lease` paths (per §11.4.41 force-push merge-first audit) where the push outcome MUST gate the next step. This is the ONLY escape — without it, every push is backgrounded.

Composes with §2.1 (multi-upstream push norm — §11.4.88 makes it non-blocking), §9.2 (data safety — backgrounded push still uses hardlinked-backup pre-op), §11.4.41 (force-push merge-first — uses `--sync-push` escape), §11.4.42 (iteration discipline — endless-loop ticks no longer waste budget on remote-network latency), §11.4.71 (fetch-before-push merge-first — applies before background push spawns), §11.4.87 (endless-loop / zero-idle — §11.4.88 is the implementation seam that makes §11.4.87(B) genuinely zero-idle for the dominant blocker class).

Pre-build gate `CM-COVENANT-114-88-PROPAGATION` enforces this anchor literal in every CLAUDE.md / AGENTS.md / QWEN.md across the canonical fleet. Pre-build gate `CM-BACKGROUND-PUSH-WIRED` verifies `commit_all.sh` releases flock before `do_push` AND spawns push via `nohup ... &` AND `push_all.sh` acquires per-remote flock. Paired §1.1 meta-test mutations strip the load-bearing literals → gates FAIL.

Canonical authority: this Constitution.md §11.4.88 in the HelixConstitution submodule (`git@github.com:HelixDevelopment/HelixConstitution.git`).

Non-compliance is a release blocker. Synchronous push (without `--sync-push` flag) is severity-equivalent to a §11.4 PASS-bluff at the project-execution layer. No escape hatch beyond `--sync-push` for §11.4.41 force-push events.

2026 06 03

§ . . — Absolute no-force-push + merge-onto-latest-main mandate (User mandate, - -)

Short tag: `absolute-no-force-push` .

Forensic anchor — verbatim user mandate (2026-06-03):

> "Any force-push is strictly forbidden! We must for every Submodule take as a base latest commit on Submodule's main (or master) branch, then on top of it carefully to merge all changes that have to be pushed! Once all merging is carefully done we perform commit and push to all Submodule's upstreams!"

The mandate — force-push is STRICTLY FORBIDDEN, with NO exception. Any force-push — `git push --force`, `git push --force-with-lease`, `git push +<ref>`, or any history-rewriting operation that overwrites a remote ref non-fast-forward — is forbidden against every repository this Constitution governs (every consuming project's main repo, this constitution submodule, every owned submodule, every nested submodule, every upstream of each). There is no escape hatch, no operator-approval path, no "after a merge-first audit" path. The merge-onto-latest-main procedure below is ALWAYS available, so a force-push is NEVER necessary; eliminating the need eliminates the operation.

The mandated 6-step integration procedure (for every repo/submodule whose local has commits to publish OR whose mirrors have diverged):

1. **Fetch all remotes** — `git fetch --all --prune --tags` against `origin` + every configured upstream; capture the output as evidence.
2. **Set the base to the LATEST commit on the canonical `main` / `master` branch** — the most-advanced mirror tip across all upstreams. The integration target is that tip, never a stale local branch point.
3. **Carefully MERGE every change that must be published on top of that base** — a union merge that preserves BOTH sides (local commits AND every remote-side commit). NEVER `-s ours` (it silently discards the remote side), NEVER a rebase/reset/ `--allow-unrelated-histories` that could drop commits (per §9 no-commit-loss). The merge commit MUST descend from every mirror tip so that every subsequent push is a clean fast-forward.
4. **Resolve every conflict carefully** so nothing is broken, lost, or corrupted on either side — no conflict markers anywhere (`grep -rn '^<<<<<<< \|^=======$\|^>>>>>>> '` returns empty across governance + source + test files), no file silently dropped, every previously-passing gate/test still passes on the integrated tree (per §11.4.4 + §11.4.40), every captured-evidence artefact still validates.
5. **Commit the merge** — stage only the intended files (NEVER `git add -A` inside a submodule with untracked build content per §11.4.30), commit the carefully-resolved merge.

6. **Push the result to ALL upstreams** — each push is a fast-forward because the merge commit descends from every mirror tip, so NO force is ever needed. If an upstream still reports non-fast-forward (a sibling landed work between step 1 and the push), do NOT force — return to step 1 for that upstream, fetch its new tip, MERGE it (union, preserve both), re-validate step 4, and re-push.

Tightens §11.4.41 / §11.4.71 / §9.2 / CONST-043 — force-push escape hatch REMOVED. §11.4.41 (Pre-Force-Push Merge-First Mandate), §11.4.71 (Pre-Push Fetch + Investigate + Integrate), §9.2 (Force-push requires explicit user authorization), and CONST-043 previously PERMITTED a force-push after a merge-first pipeline + per-operation operator approval. §11.4.113 supersedes that permitting stance: even WITH operator approval, even AFTER a clean merge-first audit, a force-push is forbidden — because the merge-onto-latest-main path in this clause is always available and always sufficient, force is never necessary and therefore never authorised. Those clauses' merge-first/fetch-first machinery REMAINS in force as the integration discipline; only their terminal "...then force-push" step is struck. §9.2's destructive-operation §9 backup discipline still applies to any non-push history operation, but a history-rewriting *push* is simply not performed.

No escape hatch. No `--force`, no `--force-with-lease`, no `+<ref>` push, no `--no-verify` to slip a forced ref past a hook, no "operator authorised the force-push" path, no "lease semantics unavailable so plain --force is OK" path. A force-push that lands on any remote is a critical defect regardless of how briefly it persisted, severity-equivalent to a §11.4 PASS-bluff at the data-safety layer (per §9).

Classification: universal (§11.4.17) — an absolute-no-force-push + merge-onto-latest-main integration discipline is platform-neutral and reusable across every repository in every consuming project; nothing here references a particular project, vendor, hardware, or region.

Composes with §2.1 (multi-upstream push is the norm — step 6 fans out to all), §9 / §9.2 (absolute data safety — §11.4.113 is the no-loss push discipline that makes force-push unnecessary), §11.4.4 (test-interrupt + clean-baseline retest — step 4 audit), §11.4.6 (no-guessing — remote state is not knowable without the step-1 fetch), §11.4.26 (constitution-submodule update workflow — its conflict-resolution step is this procedure), §11.4.37 (fetch-before-edit — step 1 generalised to fetch-before-push), §11.4.40 (full-suite retest authority for the integrated tree),

§11.4.41 (force-push merge-first — TIGHTENED: merge-first stays, the force-push step is removed), §11.4.71 (pre-push fetch + integrate — TIGHTENED the same way), §11.4.88 (background-push — the detached push still follows this fast-forward-only discipline), CONST-043 (per-operation authorization — TIGHTENED: no force-push is authorisable).

Propagation. Propagation gate `CM-COVENANT-114-113-PROPAGATION` enforces the literal anchor `11.4.113` across the consumer fleet; paired §1.1 meta-test mutation strips the literal → the gate FAILs. Recommended per-family gate `CM-NO-FORCE-PUSH-ABSOLUTE` (scans tracked scripts + hooks for any `push --force` / `push --force-with-lease` / `push +<ref>` invocation and rejects it; a §11.4.109-class PreToolUse guard blocks the force-push command class at the tool-call boundary); paired §1.1 mutation injects a `git push --force` into a tracked script → the gate FAILs. (Gate-code implementation lands as a separate work item; this anchor defines the contract.)

Canonical authority: this Constitution.md §11.4.113 in the HelixConstitution submodule. All consuming projects restate + cite via §11.4.35 inheritance.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--force`, `--force-with-lease`, `--allow-force-push`, `--force-push-authorised`, `--skip-merge-onto-main` flag exists.

1 1 8

2026 06 03

§ . . — No-commit-while-build-writes-tracked-artifacts mandate (. . -dev remediation, - -)

Short tag: `no-commit-during-artifact-write`.

Forensic anchor (genericised, 2026-06-03). The build pipeline's per-component build steps write produced artifacts (forked-app APKs) INTO tracked directories that are under version control (`prebuilt_apps/`). Running `git add -A` / a commit while those steps are mid-write stages a PARTIAL or stale artifact — a race between the build writing the file and git reading it — so the commit captures a half-written or pre-rebuild artifact. The remediation deferred the commit to BUILD-COMPLETION: the tree is quiescent (no build step writing tracked files) AND the

freshly-built artifacts are captured whole (no stale prebuilt in git). This is the artifact-write analogue of §11.4.84 (no commit while a mutation gate is in flight) at the build-output layer.

The mandate. A commit (especially `git add -A` / any broad stage) MUST NOT run while a build / packaging / generation step is actively writing artifacts INTO tracked directories. Doing so races the writer and stages a partial or stale artifact — the commit captures bytes that are mid-write or pre-rebuild, a §11.4.108 SOURCE→ARTIFACT integrity failure landed in version control. The commit MUST be deferred until the build step that writes tracked artifacts has COMPLETED, so that: (1) the working tree is quiescent at the artifact layer (no step writing a tracked file); (2) the committed artifacts are the FRESH, whole outputs of the completed build (no stale pre-rebuild artifact, no half-written file). Before committing tracked build outputs, the committer MUST verify the writing step finished (process exit, completion marker, or per-artifact mtime ≥ build-start) — a build still in flight that writes tracked dirs is a HOLD on the commit, not a race to win.

Composition with §11.4.84. §11.4.84 forbids committing while a mutation gate is in flight in the checkout (mutation residue). §11.4.121 is the BUILD-OUTPUT analogue: forbid committing while a build is in flight WRITING TRACKED ARTIFACTS (partial/stale artifact residue). Both close the same class of defect — a commit captures transient, non-final tree state — at two different write-sources (mutation experiments vs build outputs). Where build outputs land OUTSIDE version control (gitignored `out/` / `dist/`), the race does not apply; the mandate binds specifically to build steps writing into TRACKED directories.

Honest boundary (§11.4.6). "The build probably finished" is not "the build finished" — verify with a completion signal (exit code / marker / mtime), not an assumption, before committing tracked artifacts. Committing source changes that DON'T touch the build's tracked-artifact directories is fine mid-build (those files are not being written by the build); the hold applies only to the tracked dirs the in-flight build writes.

Classification: universal (§11.4.17) — deferring commit until a tracked-artifact-writing build completes is a platform-neutral commit-integrity discipline reusable by ANY project whose build writes produced artifacts into version-controlled directories; the consuming project supplies which build steps write which tracked directories and its completion-signal mechanism per §11.4.35. The PROJECT-

SPECIFIC instance (which tracked directory — e.g. a prebuilt-apps directory — and which build steps write it) is recorded in the consuming project's own governance per §11.4.35.

Composes with §11.4.6 (no-guessing — verify build completion, never assume), §11.4.30 (no-versioned-build-artifacts — artifacts that ARE legitimately tracked, like signed prebuilts, are the exact case this mandate protects from partial commit; gitignored build outputs are exempt), §11.4.58 / §11.4.103 (parallel-development — the commit/merge stage is serialized AFTER the build stage for this reason), §11.4.84 (working-tree quiescence — §11.4.121 is its build-output analogue), §11.4.88 (background-push — commit timing is decoupled from push, but the commit itself still waits for artifact-write completion), §11.4.108 (four-layer fix-verification — a partial/stale committed artifact is the ARTIFACT-layer integrity failure this prevents from entering version control), §11.4.96 (safe-parallel-work-with-long-build — committing tracked build outputs mid-build is an UNSAFE operation in its catalogue).

Propagation. Propagation gate `CM-COVENANT-114-121-PROPAGATION` enforces the literal anchor `11.4.121` across the consumer fleet; paired §1.1 meta-test mutation strips the literal → the gate FAILs. Recommended per-family gate `CM-NO-COMMIT-DURING-ARTIFACT-WRITE` (the commit wrapper refuses to stage tracked build-artifact directories while a build step writing them is in flight — checks a build-in-progress lock / completion marker); paired §1.1 mutation removes the build-in-flight check from the commit wrapper → the gate FAILs. (Gate-code implementation lands as a separate work item; this anchor defines the contract.)

Canonical authority: this Constitution.md §11.4.121 in the HelixConstitution submodule. All consuming projects restate + cite via §11.4.35 inheritance.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--commit-during-build`, `--stage-partial-artifact-OK`, `--assume-build-finished`, `--skip-build-completion-check` flag exists.

---

§11.4.179 — Corruption-isolated parallel git streams (own-.git independent clones, not shared-common-dir worktrees) (User mandate, 2026-07-04). When corruption-isolation is a requirement, parallel git work streams MUST each be an isolated repository with its OWN `.git` (own object store, own index, own lock

namespace), NOT `git worktree` checkouts sharing one common `.git`. A shared common-dir is a single point of failure: one stale `index.lock` / `.commit_all.lock` / `.push_all.lock` freezes EVERY stream at once (the observed 9-h all-track freeze, 2026-07-04), and one corrupted object store loses every stream. Each stream's `git rev-parse --git-common-dir` MUST resolve INSIDE the stream's own tree; a common-dir resolving to a shared parent is a violation. Where CoW/reflink disk-feasibility is the constraint, use the §11.4.167 reflink-clone-with-own-`.git` path — a btrfs/XFS/ZFS/APFS reflink shares extents on disk while keeping a per-stream `.git`, so isolation and disk-feasibility are BOTH satisfied. Honest boundary (§11.4.6): own-`.git` isolation contains lock/corruption blast radius — it does NOT remove the need for per-repo stale-lock reaping (§11.4.180) nor for ff-only no-force integration (§11.4.113). Composes §11.4.167 (feature work-stream lifecycle / CoW clone) / §11.4.119 (single-resource-owner) / **§11.4.176** (the multi-track work-division + exactly-once-claim + device-lock arbitration anchor) / §9.2 (data safety) / §11.4.58 (parallel PWU). Classification: universal (§11.4.17) — the consuming project supplies its reflink mechanism + clone layout per §11.4.35. Propagation gate `CM-COVENANT-114-179-PROPAGATION` (literal `11.4.179`) + recommended gate `CM-GIT-STREAMS-OWN-COMMON-DIR` (every registered parallel stream's `git rev-parse --git-common-dir` resolves inside its own tree, never a shared parent) + paired §1.1 meta-test mutation (register a stream whose common-dir points to a shared parent → gate FAILs; strip the literal → propagation gate FAILs; gate-code = separate work item). **Canonical authority:** constitution submodule `Constitution.md` §11.4.179. Non-compliance is a release blocker. No escape hatch — no `--shared-common-git`, `--worktree-when-isolation-required`, `--skip-git-isolation` flag.

§11.4.180 — Commit/push (single-writer) wrappers MUST auto-reap provably-stale locks before acquiring (User mandate, 2026-07-04). Every commit / push / single-writer wrapper MUST, before acquiring its own lock, auto-reap any PROVABLY-stale git lock — a lock whose recorded holder PID is DEAD (`kill -0` fails), OR (no PID recorded) older than a defined threshold AND with no live wrapper/git process holding that repo. The reap covers the wrapper's own lock plus the git-internal existence-based locks (`index.lock`, `HEAD.lock`, `refs/**/*.lock`) that git does NOT auto-release on holder death. It MUST NEVER remove a lock whose holder is ALIVE (removing a live-held lock corrupts a concurrent writer — §9.2). Each reap decision (REAPED / KEPT + reason) is logged as captured

evidence (§11.4.6 — liveness PROVEN via `kill -0`, never assumed). A wrapper that blocks indefinitely on a dead-holder lock (the observed 9-h freeze, 2026-07-04) is a violation. Honest boundary (§11.4.6): reaping clears dead-holder deadlock — it does NOT substitute for the own-`.git` isolation of §11.4.179 (which shrinks the blast radius from all-streams to one). Composes §11.4.84 (working-tree quiescence) / §11.4.88 (commit-lock-release-immediately + detached push) / §11.4.6 (no-guessing) / §9.2 (data safety) / §11.4.116 (evidence stream) / §11.4.179 (own-`.git` isolation companion). Classification: universal (§11.4.17) — the consuming project supplies its concrete wrapper + reap helper + stale-age threshold per §11.4.35. Propagation gate `CM-COVENANT-114-180-PROPAGATION` (literal `11.4.180`) + recommended gate `CM-WRAPPER-STALE-LOCK-AUTO-REAP` (every single-writer wrapper reaps provably-stale locks before acquiring, never removes a live-held lock, and logs each REAPED/KEPT decision) + paired §1.1 meta-test mutation (make the reaper remove a live-held lock, OR skip the reap-before-acquire step → gate FAILs; strip the literal → propagation gate FAILs; gate-code = separate work item). **Canonical authority:** constitution submodule `Constitution.md` §11.4.180. Non-compliance is a release blocker. No escape hatch — no `--skip-stale-lock-reap`, `--force-remove-live-lock`, `--block-on-dead-holder` flag.

§11.4.181 — Consistent controlled feature-branch naming: one feature/logic-group ⇒ exactly ONE canonical branch name across the main repo + all owned submodules (User mandate, 2026-07-05). Forensic anchor — verbatim user mandate (2026-07-05): "System MUST pay attention not to create multiple feat. branches with different names for one same feature or group of workable items / features! Naming MUST BE consistent and carefully controlled! This MUST BE part of the constitution too and carefully risk-free handled! No mistakes can happen! There MUST NOT be any false results, mis-alignments, inconsistency or bluff of any kind anywhere!" One feature / logical group of workable items MUST map to EXACTLY ONE canonical branch name, used IDENTICALLY on the main repo AND on every owned submodule the group touches — NEVER two (or more) differently-named branches for the same feature/group, NEVER a per-repo naming drift. The mandate (ALL hold): (1) **Single canonical name per group.** For each feature/logical-group a single canonical branch name is minted ONCE (per the §11.4.167 D scheme: `feature/<slug>` on the branch axis + `<prefix>-<base>-feat-<slug>` on the tag axis, `<slug>` lowercase snake/kebab per §11.4.29). That exact string is the branch name on the main repo AND on every touched owned

submodule; an untouched submodule stays on its base branch (it is NOT force-branched), but any submodule that IS branched for this group MUST use the identical canonical name. (2) **Single source of truth for branch names — look up, never re-invent.** The canonical name is recorded ONCE in a tracked registry and every subsequent branch creation for that group LOOKS IT UP rather than re-deriving/re-typing it. Bind this to the §11.4.176 exactly-once claim registry / the §11.4.93 workable-items DB `logic_group → branch` mapping (a `branch_name` field on the group/claim record): the claim that reserves a feature/group ALSO reserves its canonical branch name, so a second stream working the same group reads the same name instead of minting a divergent one. Creating a branch whose name does NOT match the registry's canonical name for that group — OR minting a SECOND canonical name for a group that already has one — is a §11.4.181 violation (a §11.4.6 no-guessing violation: the branch name is a looked-up FACT, never an ad-hoc re-invention). (3) **Mechanical enforcement (§11.4.75).** A pre-build/commit gate `CM-BRANCH-NAME-CONSISTENCY` verifies that every live feature branch (main repo + each owned submodule) maps to a registered feature/logical-group AND carries the identical registered canonical name across the main repo and every touched submodule; it FAILs on (a) an unregistered feature-branch name, (b) two differently-named branches bound to the same group, (c) a submodule branch whose name diverges from the group's main-repo canonical name. Paired §1.1 mutation: register a single group under two divergent branch names (or point a submodule branch at a name ≠ the group's canonical name) → the gate FAILs; restore → PASS. (4) **Risk-free reconciliation — no force, no loss (§9.2 / §11.4.113 / §11.4.84).** When a mis-named / duplicate branch is discovered, reconciling it to the canonical name MUST preserve every commit: create/checkout the canonical branch at the merge-base, MERGE the mis-named branch's commits onto it (union, no `-s ours` /rebase/reset that drops commits per §11.4.113 step 3), verify no commit lost (`git log <canonical>..<misnamed>` empty after merge) + no conflict markers, push fast-forward to all upstreams (NEVER force-push — the canonical branch descends from the mis-named tip), THEN retire the mis-named branch only after the merge is confirmed on all mirrors; delete-before-merge is forbidden. Honest boundary (§11.4.6): "the branches are probably the same feature" is NOT a determination — bind the branch to a group via the registry (claim id / logic-group id), never guess two branches are "the same feature" without the registry record. Classification: universal (§11.4.17) — a platform-neutral naming-discipline reusable across any multi-repo/multi-submodule project; the consuming project supplies its concrete registry (claim registry / workable-items DB), branch/

tag scheme, and owned-submodule set per §11.4.35. Composes §11.4.167 (feature work-stream lifecycle / D branch+tag scheme) / §11.4.176 (exactly-once claim registry — the branch-name reservation rides the claim) / §11.4.178 (track-qualified identity — a track-qualified session/lock key is distinct from the shared cross-track feature-branch name; both hold) / §11.4.179 (own-`.git` streams still share ONE canonical feature-branch name across their independent repos) / §11.4.28 (owned-submodule set) / §11.4.29 (lowercase snake/kebab slug) / §11.4.93 (workable-items DB `logic_group`) / §11.4.113 (no-force merge-onto-latest) / §11.4.84 (quiescence) / §9.2 (no-loss) / §11.4.6 (branch name is a looked-up fact) / §11.4.75 (mechanical enforcement). Propagation gate `CM-COVENANT-114-181-PROPAGATION` (literal `11.4.181` across the consumer fleet) + recommended gate `CM-BRANCH-NAME-CONSISTENCY` + paired §1.1 meta-test mutation (register one group under two divergent branch names → gate FAILs; strip the literal → propagation gate FAILs; gate-code = separate work item). **Canonical authority:** constitution submodule `Constitution.md` §11.4.181. Non-compliance is a release blocker. No escape hatch — no `--allow-divergent-branch-names`, `--skip-branch-registry`, `--rename-branch-freely`, `--per-repo-branch-name-OK`, `--two-names-one-feature-OK` flag.

§11.4.188 — Regular main→feature merge cadence: every track / feature-branch MUST FREQUENTLY merge canonical `main` into itself DURING the work, never only at the end (User mandate, 2026-07-09). Compact summary: every long-lived feature branch AND every parallel-development track (§11.4.58/§11.4.103 parallel streams; §11.4.167 feature work-streams; §11.4.176/§11.4.178/§11.4.179 multi-track) MUST regularly `git merge origin/main` (the canonical trunk/master) INTO its own branch THROUGHOUT the work — NOT only at the end — so no branch ever drifts far from main and the eventual back-merge stays a small, low-conflict, low-risk operation. GENERALISES §11.4.167(D) (trunk-merged-INTO-every-BIG-feature-STREAM, at minimum after every trunk tag / daily, `git merge` never rebase, stale-if-not-synced) from the big-work-item feature-stream lifecycle to EVERY feature branch on EVERY track — a small/medium feature branch, or any `git worktree`/CoW track that never earned a full §11.4.167 stream, is STILL bound to this cadence (the §11.4.167(D) discipline promoted to the general standalone rule; §11.4.167(D) remains its big-item specialisation). (1) **WHAT** — fetch the latest canonical `main` / `master` and merge it INTO the working branch, keeping the branch continuously close to trunk; a branch that has NOT integrated trunk within the cadence is flagged STALE (the exact long-lived-branch

conflict-storm + integration-risk anti-pattern this forbids). (2) **CADENCE** — merge trunk in FREQUENTLY: after EVERY trunk tag, at minimum DAILY while the branch is live, AND before starting any significant new chunk of work on the branch — MANY SMALL merges, never one big deferred end-of-branch merge (small frequent merges minimise the divergence + conflict surface). (3) **HOW (safe-by-construction)** — MERGE, NEVER rebase a shared/tagged/pushed branch (no commit-loss, no history rewrite — §9/§11.4.113); FETCH-FIRST every remote before the merge (§11.4.37/§11.4.71 — remote state is unknowable without a fetch, §11.4.6); take a §9.2 hardlinked pre-op backup before any LARGE / conflict-heavy / risky merge; resolve every conflict CAREFULLY — ZERO conflict markers committed, ZERO file silently dropped, union of both sides preserved (§11.4.41 step 3 / §9 no-loss); integrate ONLY when the branch is QUIESCENT (§11.4.84 — no in-flight mutation gate, no unaccounted uncommitted work); prefer running the merge + its verification in the BACKGROUND parallel to the main stream so it never stalls other work (§11.4.88/§11.4.89/§11.4.103); NEVER force-push the merged result (§11.4.113 — the merged branch fast-forwards its own mirrors). (4) **CONSISTENCY + ANTI-BLUFF** — after EVERY merge the branch MUST be PROVEN consistent, never ASSUMED: a pre-build smoke gate runs post-merge (§11.4.42 step-3 smoke) and is GREEN, a `git grep` for conflict markers (`<<<<<<<` / `=======` / `>>>>>>>`) returns EMPTY, and no commit present pre-merge on either side is lost — captured evidence per §11.4.5/§11.4.69; a merge claimed "clean" without the post-merge smoke + marker-scan + no-lost-commit check is a §11.4/§11.4.1 bluff at the integration layer. (5) **HONEST BOUNDARY (§11.4.6)** — regular trunk-integration keeps the branch MERGEABLE + low-conflict; it does NOT by itself prove the branch's own work is correct (each change still crosses §11.4.108 runtime-signature + §11.4.40 retest), and it is NOT the approval-gated back-merge TO trunk (that stays §11.4.167(I)/§11.4.40/§11.4.41 no-merge-until-approved). Classification: universal (§11.4.17) — the consuming project supplies its canonical-trunk ref name, cadence unit, post-merge smoke-gate command, and per-track worktree/CoW mechanism per §11.4.35. Composes §9/§9.2/§11.4.6/§11.4.17/§11.4.37/§11.4.41/§11.4.42/§11.4.71/§11.4.84/§11.4.88/§11.4.89/§11.4.103/§11.4.113/§11.4.167/§11.4.176/§11.4.178/§11.4.179/§11.4.181. Propagation gate `CM-COVENANT-114-188-PROPAGATION` (literal `11.4.188`) + recommended gate `CM-REGULAR-MAIN-MERGE-INTO-FEATURE` (every live feature branch / track has integrated the canonical trunk within the declared cadence; its last integration is a real `git merge` not a rebase; the post-merge smoke + zero-conflict-marker + no-lost-commit checks passed) + paired §1.1 mutation (let a live

branch go stale past the cadence, OR record a rebase of a shared branch in place of a merge, OR commit a conflict marker → the gate FAILs; gate-code = separate work item). **Canonical authority:** constitution submodule `Constitution.md` §11.4.188. Non-compliance is a release blocker. No escape hatch — no `--skip-main-merge`, `--merge-only-at-end`, `--rebase-shared-branch`, `--defer-trunk-sync`, `--no-post-merge-smoke`, `--force-push-after-merge`, `--big-items-only-trunk-sync` flag.

§ . . — Branch-structure governance: taxonomy + merge-after-live-QA + flavor/product non-merge (User mandate, - -)

Forensic anchor — verbatim operator rules (2026-07-14):

> 1. Branch taxonomy: `feat/<NAME_OF_FEATURE>`, `product/<NAME_OF_PRODUCT>`, `flavor/<NAME_OF_FLAVOR>` (extends §11.4.181 / §11.4.167(D)).
> 2. Tracks on FEATURE branches or `main` MUST — after full confirmation + validation + verification + full LIVE MANUAL TESTING (§11.4.185) — MERGE their work to `main` COMPLETELY (all submodules + main repo), ff-only, NEVER force-push (§11.4.113), no commit lost (§9.2) (extends §11.4.167(I) / §11.4.188).
> 3. Tracks with an entirely-new FLAVOR/PRODUCT do ALL work on a flavor branch AND/OR product branch (both may exist) — this product-line work does NOT merge to `main` the same way.
> 4. Commit + push ALL branches to ALL upstreams (§2.1), ff-only.

Every controlled branch in a governed repository MUST belong to one of THREE canonical taxonomy classes, and each class carries a distinct integration lifecycle:

(A) Taxonomy (extends §11.4.181 / §11.4.167(D)). A branch's PREFIX declares its class and its lifecycle:

Prefix	Class	Integration lifecycle
`feat/<NAME_OF_FEATURE>`	FEATURE	merges to `main` after full live-manual-QA (clause B)
`product/<NAME_OF_PRODUCT>`	PRODUCT line	does NOT merge to `main` the standard way (clause C)
`flavor/<NAME_OF_FLAVOR>`	FLAVOR line	does NOT merge to `main` the standard way (clause C)

The `<NAME_*>` slug is lowercase snake/kebab (§11.4.29). The canonical branch NAME for a feature / logical group is minted ONCE and used IDENTICALLY across the main repo AND every touched owned submodule (§11.4.181) — recorded once in the §11.4.176 exactly-once claim registry / the §11.4.93 workable-items `logic_group → branch` mapping and LOOKED UP, never re-invented (§11.4.6). An untouched submodule stays on its base branch. The prefix STRINGS above are the operator-canonical form; a project already using a different feature-prefix convention (e.g. `feature/…`) is compliant so long as that convention is the ONE canonical name for the group per §11.4.181 — reconciling an inconsistent set of names is the §11.4.181 risk-free MERGE-onto-canonical path (ff-only, no commit lost, NEVER a rename that drops commits), never a silent rename (§11.4.6).

(B) FEATURE / main tracks — MERGE-TO-MAIN-AFTER-LIVE-QA. A track working on a `feat/*` branch (or on `main` directly) MUST, before merging its work to `main`, satisfy IN ORDER: full confirmation + validation + verification (automated four-layer per §11.4.4(b) + §11.4.40 full-suite retest) THEN full **LIVE MANUAL TESTING** by the QA team (§11.4.185 — the FINAL human sufficiency gate; automated GREEN is necessary but NOT sufficient). Only after §11.4.185 confirmation may the merge proceed, and it MUST be COMPLETE — the main repo AND every touched owned submodule — integrated **ff-only** onto the latest `main` (§11.4.113 merge-onto-latest-main), **NEVER force-push** (§11.4.113), with **NO commit lost / NO history rewrite** (§9.2 / §11.4.41 step 3: union, zero conflict markers, zero dropped files), pushed to **ALL upstreams** (§2.1, ff-only). This is §11.4.167(I) (no-merge-until-approved) + §11.4.188 (regular trunk-integration keeps the branch mergeable) — §11.4.195 binds the §11.4.185 live-manual-QA gate as an explicit precondition of the approval-gated back-merge.

(C) FLAVOR / PRODUCT lines — DO-NOT-MERGE-TO-MAIN-THE-SAME-WAY. A track producing an ENTIRELY-NEW flavor and/or product line (a distinct downstream product, not a feature destined for the trunk) does ALL its work on a `product/<NAME>` AND/OR `flavor/<NAME>` branch (BOTH may exist for one line — a product line may carry a product branch AND a flavor branch simultaneously). This product-line work does NOT flow back into `main` via the clause-B merge — it is a divergent line with its own release lifecycle. It STILL obeys §11.4.113 (no force-push), §9.2 (no commit lost), §2.1 (push all branches to all upstreams, ff-only), and §11.4.188 (it MAY still merge `main` INTO itself to stay close to trunk — trunk-into-line is permitted and encouraged; it is the line-into-trunk direction that clause B gates and clause C withholds for product/flavor lines).

(D) Push discipline (§2.1 / §11.4.113). Commit + push ALL branches (feature, product, flavor) to ALL configured upstreams, ff-only, NEVER force-push. New branch creation is inherently ff (a new ref loses nothing, §9.2). Branch creation from an existing HEAD (`git branch <name> <sha>`) is non-disruptive (does not touch the working tree / index / other refs) and MAY proceed even in a dirty working tree under §11.4.84 provided no commit is in-flight (`.git/index.lock` absent, no live `git-commit` / `pack-objects`).

Honest boundary (§11.4.6): this anchor governs branch STRUCTURE + integration DIRECTION; it does NOT by itself prove any branch's work correct (each change still crosses §11.4.108 runtime-signature + §11.4.40 retest + §11.4.185 manual-QA).

Classification: universal (§11.4.17) — a platform-neutral branch-structure discipline reusable across any multi-track project; it references no project-specific hardware / vendor / region / asset, and is a strict generalisation of §11.4.181 (one-feature-one-canonical-branch-name) + §11.4.167(D)/(I) (feature work-stream lifecycle) + §11.4.188 (regular main→feature merge cadence) + §11.4.185 (manual-QA final confirmation). Composes §2.1 / §9 / §9.2 / §11.4.6 / §11.4.17 / §11.4.29 / §11.4.35 / §11.4.40 / §11.4.41 / §11.4.84 / §11.4.93 / §11.4.113 / §11.4.167(D)/(I) / §11.4.176 / §11.4.178 / §11.4.181 / §11.4.185 / §11.4.188 / §11.4.191. Propagation gate `CM-COVENANT-114-195-PROPAGATION` (literal `11.4.195` across the consumer fleet) + recommended gate `CM-BRANCH-TAXONOMY-CLASS` (every controlled branch's prefix is one of `feat/` | `product/` | `flavor/` | the reserved bases `main` / `master` ; a `product/` | `flavor/` branch is NOT the source of a merge onto `main` ; a `feat/` | `main` back-merge cites a §11.4.185 manual-QA confirmation) + paired

§1.1 mutation (a `product/*` branch merged onto `main`, OR a `feat/*` back-merge with no §11.4.185 citation → gate FAILs; strip the literal → propagation gate FAILs; gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.195.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--any-branch-prefix-OK`, `--merge-product-to-main`, `--skip-live-qa-before-merge`, `--force-push-branch`, `--partial-submodule-merge` flag.

§ . . — Dedicated hook-validation script: hook checks run as an explicit stage of a dedicated commit/push script, and the commit/push mechanism is ALWAYS unblocked (operator mandate, - -)

Forensic anchor (genericised): a consumer project wired its full local CI gate as an automatic git `pre-push` hook (`core.hooksPath` → hook → multi-minute build/test/sweep stage on EVERY push). The result, captured 2026-07-26: routine pushes took many minutes each, and when one gate stage inside the hook failed or stalled, the push was rejected outright — the commit/push mechanism itself became blocked, with pending commits stranded locally and no operator-usable path forward short of a forbidden `--no-verify` bypass. The gate logic was sound; wiring it as an uninterruptible implicit hook on the commit/push code path was the architectural flaw — an edge-triggered enforcer (§11.4.233) placed directly on the operator's only forward path, violating the §2 single-entrypoint discipline by making the entrypoint unusable.

The mandate, ALL of which hold:

(A) DEDICATED SCRIPT AS THE SINGLE COMMIT/PUSH ENTRYPOINT. Every consumer MUST ship a dedicated, executable, idempotent script (Lava binding: `scripts/commit-push-all.sh`; consumers bind their own path as DATA per §11.4.35) that runs, in order: (1) submodule/workspace sync, (2) commit of all pending state, (3) the hook validations as an EXPLICIT stage — the unmodified hook logic fed the real push range, Layer-cheap checks always, long gates stage-separated, (4) push to ALL configured upstreams (§2.1), (5) a closing green/clean

verification (clean status + zero unpushed commits, recursive over submodules). The script prints per-stage evidence and is safe to re-run; direct `git commit` / `git push` on the main repo ceases to be the routine workflow (§2).

(B) HOOKS MUST NOT BLOCK THE MECHANISM. Automatic git hooks wired via `core.hooksPath` MUST NOT gate routine commit/push in a way that can leave the repository uncommittable or unpushable. Disconnecting means: `core.hooksPath` unset, the hook file itself preserved UNMODIFIED in the repo, and `--no-verify` NEVER used as the routine path. Validation relocates from the implicit hook to the explicit script stage (A) and to tag/release-time gates — it is never deleted.

(C) NO GATE IS LOST. Every check the disconnected hook performed MUST remain executed — by the dedicated script's validation stage, by the tag/release gate, or by a documented scheduled run. Dropping a check silently because the hook no longer fires is a §11.4 PASS-bluff at the process layer ("the gate still exists" while nothing invokes it); the script's stage list MUST name every check it runs and every check it defers, with the deferral target recorded.

(D) ALWAYS-UNBLOCKED INVARIANT. The commit/push mechanism MUST always be able to complete. A failing validation yields a clear per-check report plus a documented remediation path — never an opaque hung or rejected push. Long gates (multi-minute build/test/sweep stages) MUST be separable from the cheap checks and skippable via an explicit, documented flag (Lava binding: `LAVA_SYNC_SKIP_CI=1`), with every skip recorded in the commit message or evidence pack so the deferred gate is tracked, never forgotten. The escape hatch is a recorded deferral, not a bypass: the skipped gate remains owed and is caught at the next full run or tag gate.

Honest boundary (§11.4.6). This anchor relocates WHERE validation executes (implicit hook → explicit script stage + release gates); it does NOT weaken any substantive gate — the Sixth/Seventh-Law-equivalent anti-bluff covenant, the tag-time full gate, and §11.4.32's sweep-after-constitution-pull all still bind at their own seams. It does NOT mandate deleting hooks (the hook file is preserved), does NOT authorise `--no-verify` as routine practice, and does NOT make a failing gate pass — it makes the mechanism reachable while the failure is remediated. Composes §2 / §2.1 (single entrypoint + all-upstreams push — this anchor keeps that entrypoint usable), §11.4.32 (sweep still runs, now as a named stage), §11.4.113 (no force-push history-rewrite — the unblocked path never rewrites),

§11.4.201 (the script is a guard: a false GREEN verification is a FAIL-bluff), §11.4.227 (amendment discipline), §11.4.233 (the commit/push path is a gated transition the anti-mess control plane reconciles).

Classification: universal (§11.4.17) — no hardware/vendor/project literal; consumers supply their script path, upstream-remote list, cheap-vs-long gate split, and skip-flag name as DATA per §11.4.35. Propagation gate `CM-COVENANT-114-234-PROPAGATION` (literal `11.4.234`) + recommended mechanism gates `CM-DEDICATED-HOOK-VALIDATION-SCRIPT` (A) / `CM-HOOKS-NEVER-BLOCK-PUSH` (B) / `CM-NO-GATE-LOST-ON-HOOK-DISCONNECT` (C) / `CM-COMMIT-PUSH-ALWAYS-UNBLOCKED` (D) + paired §1.1 mutations (wire the full gate back as an automatic blocking pre-push hook → (B) FAILs; disconnect the hook AND drop a check from every stage → (C) FAILs; a validation failure that strands commits with no remediation report → (D) FAILs; strip the literal → propagation FAILs; gate-code = separate work item, NOT claimed shipped §11.4.6/§11.4.227). No escape hatch beyond the recorded (D) deferral — no `--no-verify-routine`, `--skip-all-gates-silently`, `--delete-blocking-hook`, `--push-without-any-validation` flag.

2026 08 15

§ . . — Fail-closed-on-dangerous-combination for mutating / credential surfaces (research-derived, - -)

Full anchor per Phase 3 landing brief §11.4.252. Every code path COMBINING ≥ 2 dangerous capabilities MUST FAIL CLOSED — refuse operation unless every capability's precondition verifiably satisfied — NEVER fail open on ambiguous/missing/unresolvable input. Security-engineering generalisation of §11.4.201 guard-honesty applied to code paths where false-negative blast radius dwarfs operator inconvenience of false-positive. DANGEROUS-COMBINATION TAXONOMY: (1) MUTATION of shared resource (DB / filesystem / container state / config), (2) UNTRUSTED INPUT (user string / network bytes / external env var / unverified file), (3) CREDENTIAL ACCESS (secret read / authenticated API call / signing key use), (4) EXTERNAL SIDE EFFECT (email / webhook / MQ publish / payment endpoint), (5) SHELL / EXEC (subshell / binary with user-influenced args / template eval with user values), (6) IRREVERSIBLE (delete / drop / truncate — §11.4.113 force-push already absolutely forbidden). Any path combining ≥ 2 = dangerous-combination, MUST fail closed. INVARIANT: (1) VERIFY every precondition (authorization + input

11 4 252

shape + credential presence + target identity); (2) REFUSE when ANY unverifiable with ERROR EXIT naming specific unresolved precondition; (3) EMIT captured evidence per §11.4.5 of refusal; (4) NEVER default to allow/proceed/retry-with-relaxed on ambiguity — §11.4.101 conservative-safe default = REFUSE. FAIL-OPEN ANTI-PATTERNS refused: `catch { /* ignore */ }`, `credential = credential || default`, `if (!valid) { log("warn"); return proceed(input); }`, `target = user_input || default_target`, unbounded `retry { dangerous(); }`. Distinct from graceful degradation. [MATERIAL-THIN: 6-class dangerous-combination taxonomy is a §11.4.17 generalisation of the fail-closed pattern's shape applied to specific dangerous surfaces; closed-set formalisation, not literal per-class enumeration.] Composes §11.4.5 / §11.4.6 / §11.4.10 (credentials specialisation) / §11.4.66 (operator escalation when refuse stuck) / §11.4.101 / §11.4.113 (max instantiation) / §11.4.194 / §11.4.201 / §1.1. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-252-PROPAGATION` (literal `11.4.252`) + recommended gate `CM-DANGEROUS-COMBINATION-FAIL-CLOSED` + paired §1.1 mutation.

Canonical authority: constitution submodule `Constitution.md` §11.4.252. Non-compliance is a release blocker. No escape hatch — no `--fail-open-on-error`, `--swallow-exception`, `--default-missing-credential`, `--proceed-on-ambiguous-input`, `--retry-until-succeeds` flag.

2026 08 15

§ . . — Idempotency under retry + DB-level durable uniqueness guard (research-derived, - -)

Full anchor per Phase 3 landing brief §11.4.253. Every retryable operation (network calls with auto-retry, at-least-once MQ consumers, cron with overlapping schedules, HTTP endpoints with client-retry, DB transactions with retry-on-serialisation-failure, deploy pipelines with rerun capability) MUST BE IDEMPOTENT: N executions produce same observable end state as one. Idempotency ENFORCED at LOWEST LAYER where "same end state" is durably provable — typically DATABASE UNIQUENESS CONSTRAINT on caller-supplied idempotency key, NEVER only application-layer check-then-insert (races). Prevents DOUBLE-CHARGE / DOUBLE-EMAIL / DOUBLE-DELETE / DOUBLE-EFFECT bugs. IDEMPOTENCY-KEY DISCIPLINE: every retryable op accepts (or generates + passes forward)

unique identifier naming THIS logical op; persistence layer has UNIQUE INDEX / CONSTRAINT on key column; second attempt REFUSED by DB at durable level not application logic reading-then-writing; refusal caught by caller as "duplicate — already succeeded" = SUCCESS. ANTI-PATTERNS: (i) check-then-insert without unique index races; (ii) app-level dedup table read without spanning-transaction; (iii) key scoped to app instance (in-memory not shared); (iv) effect persisted BEFORE key (crash between = double-effect on retry); (v) no key at all + "the effect is 'set X to V' so redoing is fine" (safe for pure sets, NOT for INSERTs / MQ emissions / external-system side effects); (vi) idempotency-key expires — converts to time-bounded-idempotent = §11.4.6 mis-labelling if caller expects unbounded. CHAOS VERIFICATION per §11.4.85: (1) SIMULTANEOUS RETRIES (same key from 2 concurrent callers), (2) MID-OPERATION CRASHES (kill receiver between "check exists" and "insert effect"), (3) RETRY AFTER APPARENT SUCCESS (caller believes fail, receiver persisted, retry MUST recognise duplicate). External-system side effects (Stripe/PayPal charging card + local DB write): idempotency key MUST pass through to external; if external does not support, op is not fully idempotent — label honestly per §11.4.6. Composes §11.4.5 / §11.4.6 / §11.4.85 / §11.4.201 / §11.4.252 / §1.1. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-253-PROPAGATION` (literal `11.4.253`) + recommended gates `CM-IDEMPOTENCY-DB-LEVEL-UNIQUE-GUARD` + `CM-IDEMPOTENCY-CHAOS-VERIFIED` + paired §1.1 mutation.

Canonical authority: constitution submodule `Constitution.md` §11.4.253. Non-compliance is a release blocker. No escape hatch — no `--check-then-insert-OK`, `--application-dedup-suffices`, `--skip-idempotency-chaos-test`, `--claim-idempotent-without-key`, `--expire-idempotency-marker-silently` flag.
